W1

Class Diagrams & Design Principles (SOLID)

Data Structures & Object-Oriented Programming



Table of Contents

- UML Class Diagram
- Relationships
- SRP (Single Responsibility Principle)
- OCP (Open/Closed Principle)

บทนำ

- ตลอด 5 สัปดาห์ที่ผ่านมา
  - สัปดาห์ที่ 1 คือ Class และ Object
  - สัปดาห์ที่ 2 คือ Encapsulation
  - สัปดาห์ที่ 3 คือ Inheritance
  - สัปดาห์ที่ 4 คือ Polymorphism
  - สัปดาห์ที่ 5 คือ Association, Aggregation, Composition
  - ทุกเครื่องมือเหล่านี้ตอบคำถามเดียวกันคือ “จะเขียน code ให้ทำงานได้อย่างไร”
- แต่สัปดาห์นี้จะตอบคำถามที่ต่างไป คือ “จะเขียน code ให้ ‘ดี’ ได้อย่างไร”
  - เข้าใจง่าย (คนอื่นอ่าน code แล้วรู้ทันทีว่าแต่ละส่วนทำหน้าที่อะไร)
  - แก้ไขได้โดยไม่กระทบส่วนอื่น (แก้ปัญหาจุดหนึ่งแล้วไม่ทำให้จุดอื่นพัง)
  - ขยายได้โดยไม่ต้องรื้อ code เดิม (เพิ่ม feature ใหม่ได้โดยแทบไม่ต้องแตะ code ที่ทำงานดีอยู่แล้ว)
- ความสัมพันธ์ระหว่างสองสิ่งนี้คือ Class, Encapsulation, Inheritance, Polymorphism คือ เครื่องมือ ส่วน SOLID Principles ที่จะเรียนสัปดาห์นี้คือ หลักการเลือกใช้เครื่องมือให้ถูกจุด

“ทำงานได้” ยังไม่เพียงพอ

```python
class UserManager:
    def __init__(self, db_connection):
        self.db = db_connection
        self.users = []

    def add_user(self, name: str, email: str) -> None:
        # ตรวจสอบ email
        if "@" not in email or "." not in email.split("@")[-1]:
            print("Email ไม่ถูกต้อง")
            return

        # บันทึกลง database
        self.db.execute(f"INSERT INTO users VALUES ('{name}', '{email}')")

        # ส่ง welcome email
        print(f"Sending welcome email to {email}...")

        # สร้าง activity log
        with open("log.txt", "a") as f:
            f.write(f"User added: {name}\n")

        self.users.append({"name": name, "email": email})

    def generate_html_report(self) -> str:
        rows = "".join(f"<tr><td>{u['name']}</td><td>{u['email']}</td></tr>"
                       for u in self.users)
        return f"<table>{rows}</table>"

    def generate_csv_report(self) -> str:
        return "\n".join(f"{u['name']},{u['email']}" for u in self.users)
```

class ต่อไปนี้ทำงานได้อย่างสมบูรณ์ ไม่มี bug ไม่มี syntax error รันแล้วได้ผลลัพธ์ถูกต้องทุกประการ

ปัญหาของ class นี้ไม่ได้อยู่ที่ “มันทำงานตอนนี้ได้ไหม” แต่อยู่ที่ “มันจะยังทำงานได้ดีไหม เมื่อ requirement เปลี่ยน”

ในการทำงานจริง requirement เปลี่ยนอยู่ตลอดเวลาอย่างหลีกเลี่ยงไม่ได้

สถานการณ์ที่ 1 — ต้องเปลี่ยนกฎการตรวจสอบ email

สมมติบริษัทต้องการเพิ่มกฎว่า email ต้องมาจาก domain ที่อนุญาตเท่านั้น (เช่น @company.com หรือ @gmail.com)
สิ่งที่ต้องทำคือเปิด UserManager แล้วแก้ไข logic ใน add_user() ตรงส่วนตรวจสอบ email

```
def add_user(self, name: str, email: str) -> None:
    # ต้องแก้ตรงนี้
    allowed_domains = ["company.com", "gmail.com"]
    domain = email.split("@")[-1] if "@" in email else ""
    if domain not in allowed_domains:
        print("Email ไม่ถูกต้อง")
        return
    ...
```

การแก้ไขครั้งนี้เกิดขึ้น ภายใน method เดียวกัน กับที่บันทึก database ส่ง email และเขียน log ซึ่งหมายความว่า เวลาแก้ไขส่วนนี้ ต้องอ่านและเข้าใจทั้ง method เพื่อไม่ให้กระทบส่วนอื่นโดยไม่ตั้งใจ

สถานการณ์ที่ 2 — ต้องเปลี่ยนจาก log file เป็น log ลง database

สมมติทีมตัดสินใจว่า log ควรเก็บใน database แทนไฟล์ text เพื่อให้ query ย้อนหลังได้ง่ายขึ้น
สิ่งที่ต้องทำคือเปิด UserManager อีกครั้ง แล้วแก้ไขส่วนที่เขียน log

```
def add_user(self, name: str, email: str) -> None:
    ...
    # ต้องแก้ตรงนี้ — จากเขียนไฟล์ เป็นเขียน database
    self.db.execute(f"INSERT INTO logs VALUES ('User added: {name}')")
    ...
```

สังเกตว่า class เดิม ถูกเปิดแก้ไขเป็นครั้งที่สองแล้ว ทั้งที่เหตุผลของการเปลี่ยนแปลงครั้งนี้ไม่มีอะไรเกี่ยวข้องกับสถานการณ์ที่ 1 เลย แต่เพราะทุกอย่างอยู่ใน method เดียวกัน จึงต้องแก้ไขไฟล์เดิม จุดเดิมอีกครั้ง

สถานการณ์ที่ 3 — ต้องเพิ่ม PDF report format

สมมติฝ่ายบัญชีต้องการรายงานในรูปแบบ PDF เพิ่มเติมจาก HTML และ CSV ที่มีอยู่
สิ่งที่ต้องทำคือเปิด UserManager เป็นครั้งที่สาม แล้วเพิ่ม method ใหม่

```
def generate_pdf_report(self) -> str:
    # ต้องเพิ่ม method ใหม่ตรงนี้
    ...
```

การเพิ่ม method ใหม่ดูเหมือนจะปลอดภัยกว่าสองสถานการณ์แรกเพราะไม่ได้แก้ของเดิม แต่ก็ยังคงเป็นการเปิด class เดียวกัน อีกครั้ง และ class นี้ก็ยิ่งใหญ่ขึ้นเรื่อยๆ ตามจำนวน format ที่เพิ่มเข้ามา

ต้นทุนที่จ่ายจริง

- UserManager ถูกเปิดแก้ไขทั้งสามครั้ง ทั้งที่แต่ละครั้งมีเหตุผลที่ไม่เกี่ยวข้องกันเลย
  - ครั้งแรกคือกฎทางธุรกิจเปลี่ยน
  - ครั้งที่สองคือการตัดสินใจด้าน infrastructure เปลี่ยน
  - ครั้งที่สามคือความต้องการของผู้ใช้งานเปลี่ยน
- class เดียวที่มี “เหตุผลที่ต้องเปลี่ยน” อยู่หลายอย่างพร้อมกัน แบบนี้ ก่อให้เกิดต้นทุนที่จับต้องได้จริงสามด้าน
  - สร้าง Bug : แม้จะตั้งใจแก้แค่ส่วนเดียว แต่ก็มีความเสี่ยงที่จะพิมพ์ผิดหรือเข้าใจผิดในส่วนอื่นที่อยู่ใกล้กัน เพราะทุกอย่างอยู่ปะปนกันใน method เดียว
  - ใช้เวลาทำความเข้าใจ : ก่อนจะแก้ไขอะไรก็ตาม ผู้พัฒนาต้องอ่านทั้ง method เพื่อแยกให้ออกว่าส่วนไหนเกี่ยวกับ email validation ส่วนไหนเกี่ยวกับ database ส่วนไหนเกี่ยวกับ logging
  - ทดสอบยาก : ถ้าต้องการเขียน unit test สำหรับ “ตรวจสอบว่า email validation ทำงานถูกต้อง” ก็จะทดสอบแยกส่วนไม่ได้ เพราะการเรียก add_user() จะพ่วงเอาการเขียน database และการเขียนไฟล์ log ติดมาด้วยเสมอ

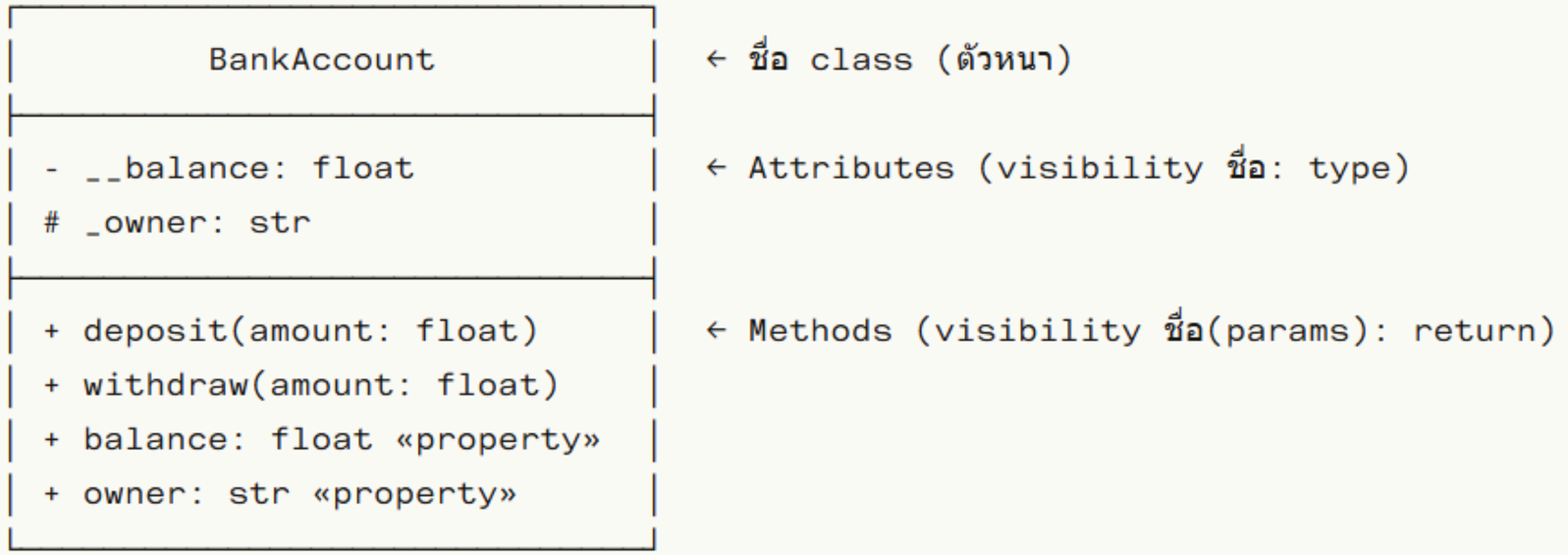
UML Class Diagram

ทำไมต้องมี “ภาษากลาง” ในการออกแบบ

- ทีมพัฒนา 4 คน กำลังช่วยกันสร้างระบบจองห้องพัก
- หัวหน้าทีมอธิบายด้วยคำพูดว่า “เรามี Room กับ Booking แล้ว Room หนึ่งห้องจะมีได้หลาย Booking นะ ส่วน Customer ก็ผูกกับ Booking อีกที”
    - คำอธิบายนี้ฟังดูชัดเจน แต่เมื่อสมาชิกแต่ละคนแยกย้ายไปเขียน code จริง คำถามที่ตอบไม่ได้จากคำพูดเพียงอย่างเดียวจะโผล่ขึ้นมาทันที เช่น
        - “Room หนึ่งห้อง มีได้กี่ Booking พร้อมกัน”
        - “ถ้าลบ Room ออกจากระบบ Booking ที่ผูกอยู่จะหายไปด้วยไหม หรือยังคงอยู่”
        - “Customer เข้าถึงข้อมูล Booking ได้โดยตรง หรือต้องผ่าน Room เท่านั้น”
- การสื่อสารด้วยคำพูดหรือข้อความบรรยาย “กำกวม” สมาชิกแต่ละคนอาจตีความต่างกัน
- UML (Unified Modeling Language) ถูกสร้างขึ้นมาเพื่อแก้ปัญหานี้โดยเฉพาะ มันเป็นมาตรฐานสากลสำหรับวาด diagram อธิบายระบบซอฟต์แวร์ ที่ทุกสัญลักษณ์มีความหมายตายตัวเพียงหนึ่งเดียว
- ในการออกแบบ OOP ใช้ class diagram บ่อยสุด ซึ่งแสดง class, attributes, methods, และความสัมพันธ์ระหว่างกัน
- ประโยชน์ที่ได้ คือ ทีมสามารถพบข้อขัดแย้งในการออกแบบได้ก่อนที่จะลงมือเขียน code เพราะเมื่อวาด diagram แล้ว คำถามอย่าง “Room หนึ่งห้องมีได้กี่ Booking” จะตอบได้ทันที

โครงสร้างของ Class Box

- หน่วยพื้นฐานที่สุดของ class diagram คือ class box ซึ่งแบ่งเป็นสามช่องเสมอ



- ช่องบนสุด คือ ชื่อ class เขียนด้วยตัวหนา
- ช่องกลาง คือ รายการ attributes ทั้งหมดของ class
  - เขียนในรูปแบบ ชื่อ: type
- ช่องล่างสุด คือ รายการ methods
  - เขียนในรูปแบบ ชื่อ(parameters): return type

แถว + balance: float «property» ในช่อง methods
แม้ว่า balance จะดูเหมือน attribute แต่ในทาง UML มันถูกจัดเป็น method เพราะเบื้องหลังคือ @property ที่ Python เรียก method จริงๆ อยู่ คำว่า «property» ซึ่งเป็นป้ายกำกับพิเศษที่ใช้บอกว่า method นี้มีพฤติกรรมพิเศษต่างจาก method ทั่วไป

Visibility Symbols — แปลจาก Python สู่ UML

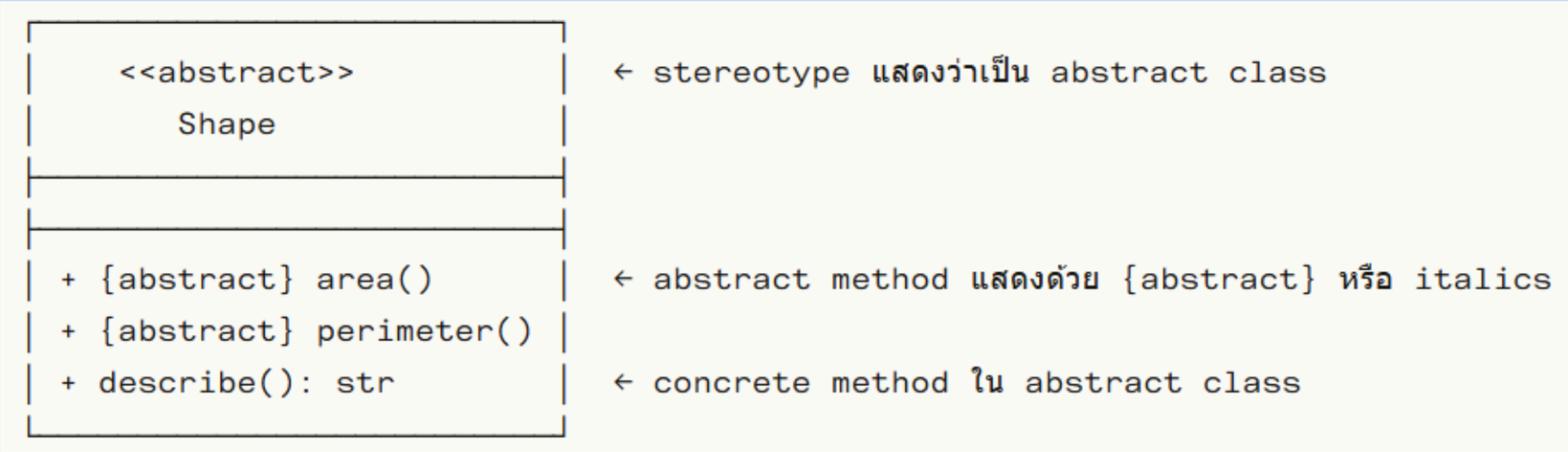
- สามสัญลักษณ์นี้ปรากฏหน้าทุก attribute และ method เสมอ และตรงกับ Encapsulation

Symbol	ความหมาย	Python ที่ตรงกัน
`+`	Public	`self.attr`
`-`	Private	`self.__attr` (name mangling)
`#`	Protected	`self._attr`

- **เมื่อนิสิตเห็น** diagram **ที่มี** __balance: float **ก็สามารถอนุมานได้ทันทีเลยว่า** attribute **ตัวนี้ เข้าถึงจากภายนอก** class **โดยตรงไม่ได้ และต้องมี** method **หรือ** property **คอยควบคุมการเข้าถึงเสมอ**

Abstract Class และ Interface ใน UML

- UML มีสัญลักษณ์เฉพาะสำหรับแสดงว่า class ใดเป็น Abstract Class



- <<abstract>> เขียนไว้เหนือชื่อ class เพื่อบอกว่านี่คือ Abstract Class ที่สร้าง object ตรงๆ ไม่ได้
- area() มีป้ายกำกับ {abstract} กำกับไว้ ซึ่งตรงกับ @abstractmethod ที่บังคับให้ subclass ต้อง implement เอง
- describe(): str ไม่มี ป้ายกำกับ {abstract} เพราะเป็น concrete method ที่มี logic จริงอยู่แล้ว

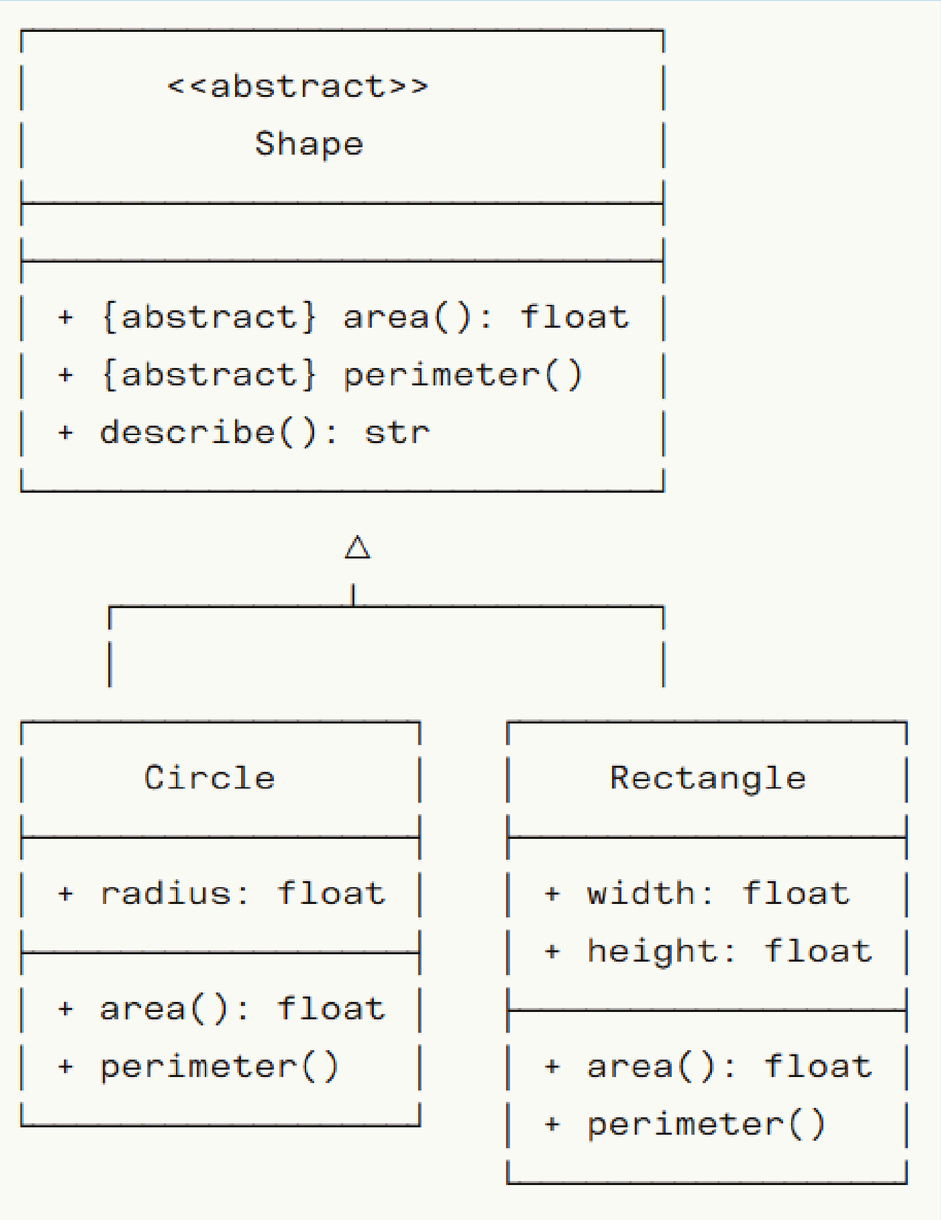
ความสอดคล้องระหว่าง diagram กับ code แบบนี้คือเหตุผลที่ UML มีประโยชน์จริง
เพราะนิสิตที่อ่าน diagram เพียงอย่างเดียว จะสามารถเดา code ที่ต้องเขียนได้เกือบทั้งหมด

วิธีอ่าน Diagram แบบทีละขั้น

เมื่อเจอ class diagram ที่ไม่คุ้นเคย ให้อ่านตามลำดับนี้เสมอเพื่อไม่ให้พลาดรายละเอียดสำคัญ

- ขั้นที่ 1 — ดูชื่อ class ก่อนเสมอ เพื่อรู้ทันทีว่านี่คือ class ปกติหรือ Abstract Class
  - ถ้ามี <<abstract>> แสดงว่าห้ามสร้าง object จาก class นี้โดยตรง
- ขั้นที่ 2 — ไล่อ่าน attributes ทีละบรรทัด พร้อมสังเกต visibility symbol ของแต่ละตัว เพื่อรู้ว่าอะไรเข้าถึงได้จากภายนอก อะไรต้องผ่าน method เท่านั้น
- ขั้นที่ 3 — ไล่อ่าน methods ทีละบรรทัด พร้อมสังเกตว่ามีตัวไหนมี {abstract} กำกับหรือไม่ ถ้ามีแปลว่า class นี้ (หรือ subclass ของมัน) ต้อง implement method นั้น
- ขั้นที่ 4 — มองหาเส้นเชื่อมที่ออกจาก class box เพื่อดูว่า class นี้มีความสัมพันธ์กับ class อื่นแบบใด

ตัวอย่าง



- Shape เป็น Abstract Class ที่มี area() และ perimeter() เป็น abstract method และมี describe() เป็น concrete method ที่ใช้ร่วมกันได้ทันที
- ส่วน Circle และ Rectangle เป็น subclass ที่มีเส้นลูกศรหัวกลวงชี้กลับไปหา Shape และทั้งคู่มี attribute เฉพาะตัวของตนเอง พร้อม implement area() และ perimeter() เป็นของตัวเอง

วิธีวาด Diagram จาก Code ที่มีอยู่แล้ว

ทำตามขั้นตอนนี้เมื่อต้องวาด diagram จาก code

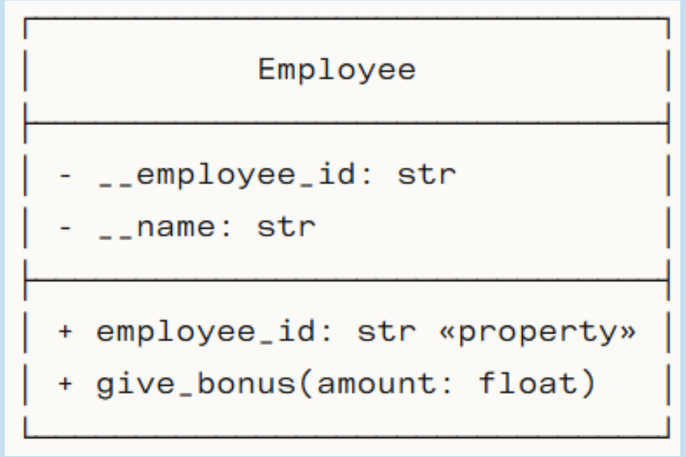
1. สร้าง class box เปล่าสำหรับทุก class ที่เจอ เขียนแค่ชื่อไว้ก่อน ยังไม่ต้องใส่รายละเอียด
2. เติม attributes จาก __init__ ไล่ดูทุกบรรทัดที่มี self.xxx = ... แล้วแปลงเป็นแถวในช่อง attribute โดยแปลง self.__balance เป็น - __balance และแปลง self._owner เป็น # _owner
3. เติม methods จากทุก method ที่ไม่ใช่ __init__ รวมถึง @property ที่ต้องเขียนกำกับด้วย «property»
4. ตรวจว่า class ใดสืบทอดจาก class ใด จาก syntax class SavingsAccount(BankAccount): แล้ววาดลูกศรหัวกลวงจาก child ไปยัง parent
5. ตรวจว่า class ใดใช้งาน class อื่นเป็น attribute หรือ parameter เพื่อวาดเส้นความสัมพันธ์ที่เหลือ

ตัวอย่าง

```python
class Employee:
    def __init__(self, employee_id: str, name: str, salary: float):
        self.__employee_id = employee_id
        self.__name = name
        self.salary = salary

    @property
    def employee_id(self) -> str:
        return self.__employee_id

    def give_bonus(self, amount: float) -> None:
        ...
```



Relationships

6 ระดับความสัมพันธ์ระหว่าง Class

- ความสัมพันธ์ระหว่าง class ในโลกจริงไม่ได้มีความ “เข้มแข็ง” เท่ากันหมด
  - บาง class แค่ “ยืมใช้” อีก class ชั่วคราวแล้วจบไป
  - บาง class “เป็นส่วนหนึ่ง” ของอีก class แต่ยังแยกตัวออกไปมีชีวิตอิสระได้
  - บาง class “ผูกชะตากรรม” กับอีก class อย่างสมบูรณ์ ถ้าฝ่ายหนึ่งตายอีกฝ่ายก็ต้องตายตาม
- ความแตกต่างของ ระดับความแน่น (coupling) นี้ส่งผลโดยตรงต่อวิธีเขียน code จริง
  - object หนึ่งควรถูกสร้างที่ไหน ถูกทำลายเมื่อไร และส่งต่อกันอย่างไร
- การใช้เส้นเดียวสื่อทุกอย่างจะทำให้ข้อมูลสำคัญเหล่านี้หายไป และทีมพัฒนาต้องกลับไปถามหรือเดาเองอยู่ดี ซึ่งขัดกับเป้าหมายของการมีภาษากลางตั้งแต่ต้น

1. Inheritance (is-a) — สืบทอดคุณสมบัติ



- child class “เป็น” parent class ประเภทหนึ่ง
- แสดงด้วยลูกศรหัวกลวง (△) ชี้จาก child ไปยัง parent เสมอ
- Inheritance เป็นความสัมพันธ์ระดับ class กับ class ไม่ใช่ระดับ object กับ object
  - เกิดขึ้นตั้งแต่ตอนนิยาม class และแก้ไขระหว่างการทำงานของโปรแกรมไม่ได้
  - object ที่เกิดจาก SavingsAccount จะเป็น SavingsAccount ไปตลอด ไม่สามารถเปลี่ยนให้กลายเป็น BankAccount เฉยๆ ระหว่างทางได้
- ประโยชน์ที่ได้จริง คือการนำพฤติกรรมและข้อมูลของ parent มาใช้ซ้ำได้ทันทีโดยไม่ต้องเขียนใหม่
- Inheritance เป็นความสัมพันธ์ที่แน่นที่สุด เพราะ child ผูกติดกับโครงสร้างภายในของ parent อย่างลึกซึ้ง
- การใช้ Inheritance ผิดที่ จะสร้างปัญหาที่แก้ไขยากในระยะยาว จึงต้องผ่าน is-a test ให้แน่ใจก่อนใช้เส้นนี้เสมอ

2. Realization (implements) — ปฏิบัติตามสัญญา

```
Shape (abstract) ◁- - - - - - Circle
```

- class ที่เป็นรูปธรรม (concrete class) รับสัญญาจาก Abstract Class หรือ In
- แสดงด้วย เส้นประ หัวกลวงชี้จาก concrete class ไปยัง abstract class
- จุดสังเกตสำคัญที่แยก Realization ออกจาก Inheritance คือ เส้นทึบกับเส้นประ
  - Inheritance ใช้เส้นทึบเพราะเป็นการสืบทอด “ทั้งโครงสร้าง” มาจริงๆ
  - Realization ใช้เส้นประเพราะเป็นเพียงการ “รับปากว่าจะทำตามสัญญา” (implement abstract method ให้ครบ) ไม่ได้รับโครงสร้างภายในมาทั้งหมด
- ประโยชน์ คือ diagram สื่อสารได้ทันทีว่า class ใดเป็นเพียง “การ implement ตามสัญญา” ซึ่งต่างจาก “การรับโครงสร้างมาใช้ต่อ”
  - สำคัญเวลาต้องประเมินผลกระทบของการแก้ไข parent เพราะ Realization กระทบ child น้อยกว่า Inheritance

3. Association (uses) — ความสัมพันธ์ทั่วไป

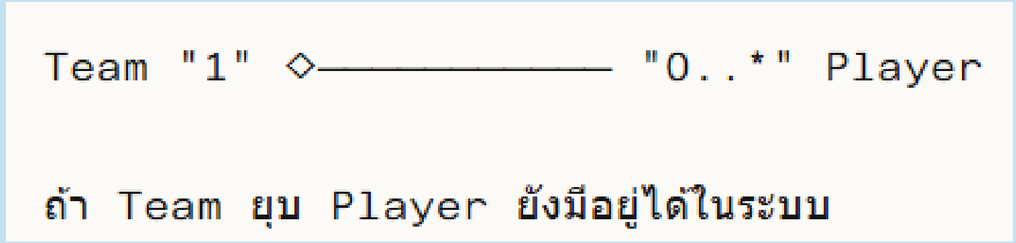
```
Customer "1" ———————— "0..*" Order
```

- class หนึ่ง “ใช้งาน” อีก class หนึ่ง โดยไม่มี ownership ต่อกัน
- แสดงด้วยเส้นทึบธรรมดาไม่มีหัวลูกศรพิเศษ (หรือบางครั้งมีลูกศรธรรมดาบอกทิศทางการเข้าถึง)
- ตัวเลขที่กำกับปลายเส้นแต่ละด้านเรียกว่า multiplicity บอกว่า object ฝั่งนั้นมีได้กี่ตัวในความสัมพันธ์นี้
  - Customer "1" ———— "0..*" Order อ่านว่า Customer หนึ่งคนสามารถมี Order ได้ตั้งแต่ 0 ถึงไม่จำกัดจำนวน (0..*)
- สัญลักษณ์ multiplicity ที่พบบ่อยมีดังนี้

Multiplicity	ความหมาย
`1`	ต้องมีพอดี 1
`0..1`	มีหรือไม่มีก็ได้ แต่ถ้ามีต้องไม่เกิน 1
`0..*` หรือ `*`	มีได้ตั้งแต่ 0 ถึงไม่จำกัด
`1..*`	ต้องมีอย่างน้อย 1 ถึงไม่จำกัด

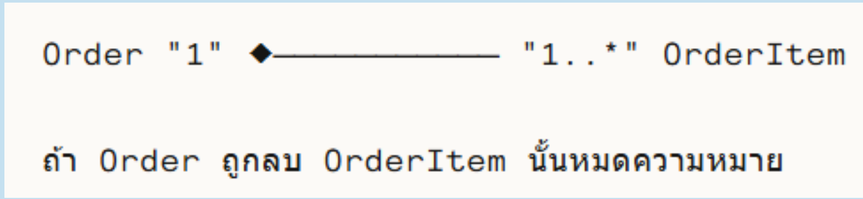
ประโยชน์ของการระบุ multiplicity ให้ชัดเจน คือบังคับให้ทีมตัดสินใจ business rule ตั้งแต่ตอนออกแบบ
เช่น “ลูกค้าหนึ่งคนสั่งได้กี่ Order พร้อมกัน”

4. Aggregation (has-a, อ่อน) — “มี” แบบที่ส่วนประกอบเป็นอิสระ



- class หนึ่ง “มี” อีก class หนึ่งเป็นส่วนประกอบ แต่ส่วนประกอบนั้น ดำรงอยู่ได้อย่างอิสระ แม้ไม่มี “เจ้าของ” แล้วก็ตาม
- แสดงด้วย เพชรกลวง (◇) อยู่ฝั่ง “เจ้าของ” (Team)
- Team กับ Player — นักฟุตบอลคนหนึ่งเป็นสมาชิกของทีมได้ แต่ถ้าทีมนั้นถูกยุบไป นักฟุตบอลคนนั้นก็ยังคงมีตัวตนอยู่ในระบบ (อาจย้ายไปทีมอื่น หรือกลายเป็น free agent)
  - Player ไม่ได้หมดความหมายไปพร้อมกับ Team

5. Composition (has-a, แน่น) — “ประกอบด้วย” แบบที่ส่วนประกอบพึ่งพาเจ้าของ



- class หนึ่ง “ประกอบด้วย” อีก class หนึ่งอย่างแนบแน่น จนส่วนประกอบนั้น ดำรงอยู่ไม่ได้เลยหากไม่มีเจ้าของ
- แสดงด้วย เพชรทึบ (◆) อยู่ฝั่งเจ้าของ (Order)
- Order กับ OrderItem — รายการสินค้าหนึ่งบรรทัดใน order (เช่น “เสื้อยืดสีน้ำเงิน จำนวน 2 ตัว”) มีความหมายอยู่ได้ก็ต่อเมื่อมันเป็นส่วนหนึ่งของ order ใบใดใบหนึ่งเท่านั้น
  - ถ้า order นั้นถูกยกเลิกหรือลบทิ้ง OrderItem นั้นก็ไม่มีความหมายอะไรอีกต่อไป ไม่มีใครไปอ้างอิงถึงมันได้อีกในฐานะ item เดี่ยวๆ ที่ลอยอยู่

เปรียบเทียบ Association, Aggregation, Composition

ความสัมพันธ์	สัญลักษณ์	ระดับ ownership	lifetime ของ “part”
Association	เส้นทึบธรรมดา	ไม่มี ownership	อิสระสมบูรณ์ ไม่เกี่ยวกับอีกฝ่ายเลย
Aggregation	◇ เพชรกลวง	มี “แต่หลวม”	อิสระ แม้เจ้าของหายไป
Composition	◆ เพชรทึบ	มีเต็มรูปแบบ	ผูกติดกับเจ้าของ ตายไปด้วยกัน

- ถ้าเลือกใช้ผิด จะส่งผลต่อการออกแบบ code จริง
  - ถ้าออกแบบเป็น Composition ทั้งที่ควรเป็น Aggregation จะทำให้ไม่สามารถให้นักเตะคนเดียวกันย้ายทีมได้ เพราะระบบออกแบบไว้ให้ Player ผูกติดกับทีมเดียวตลอดไป
  - ในทางกลับกัน ถ้าออกแบบเป็น Aggregation ทั้งที่ควรเป็น Composition (เช่น อนุญาตให้สร้าง OrderItem ลอยๆ แล้วค่อยเอาไปผูกกับ Order ทีหลัง) จะเปิดช่องให้เกิด OrderItem ที่ไม่ได้ผูกกับ order ใดเลย ทำให้ข้อมูลไม่สมเหตุสมผล

ตารางสรุปสัญลักษณ์ทั้งหมด

UML Symbol	ความสัมพันธ์	ตัวอย่าง
◁——	Inheritance	SavingsAccount extends BankAccount
◁- - -	Realization	Circle implements abstract Shape
———	Association	Customer places Order
◇———	Aggregation	Team has Player (Player อิสระ)
◆———	Composition	Order has OrderItem (Item ไม่อิสระ)

SRP (Single Responsibility Principle)

SRP (Single Responsibility Principle)

- Robert C. Martin => “A class should have only one reason to change.”
- SRP ไม่ได้หมายความว่า class ต้องมี method แค่ตัวเดียว และไม่ได้หมายความว่า class ต้องมี method จำนวนน้อยๆ เท่านั้น
  - สิ่งที่ SRP ควบคุมจริงๆ คือ จำนวนเหตุผลที่ class จะต้องถูกแก้ไข ไม่ใช่จำนวน method
- class ที่มี 10 methods ที่เกี่ยวกับการจัดการสินค้าคงคลัง (เพิ่มสินค้า ลดสินค้า ตรวจสอบสต็อกต่ำ คำนวณมูลค่ารวม ฯลฯ) ยังคงผ่าน SRP อย่างสมบูรณ์
  - เพราะทุก method มี “เหตุผลในการเปลี่ยนแปลง” เดียวกันคือ กฎการจัดการสินค้าคงคลังเปลี่ยน
- ในทางกลับกัน class ที่มีเพียง 2 methods คือ calculate_tax() กับ send_email() กลับ ละเมิด SRP
  - เพราะทั้งสอง method มีเหตุผลในการเปลี่ยนที่ต่างกันโดยสิ้นเชิง
    - calculate_tax() เปลี่ยนเมื่อกฎหมายภาษีเปลี่ยน
    - send_email() เปลี่ยนเมื่อผู้ให้บริการ email หรือรูปแบบข้อความเปลี่ยน
    - คนที่ตัดสินใจเป็นคนละกลุ่มในองค์กรจริง (ฝ่ายบัญชี กับ ฝ่าย IT/Marketing)

แนวทางในการตรวจสอบ

- คำถามเพื่อตรวจสอบ SRP คือ “ถ้า requirement เปลี่ยน จะต้องแก้ class นี้ด้วยเหตุผลกี่อย่าง?”
  - ถ้าคำตอบคือ 1 แสดงว่า class นั้นผ่าน SRP แล้ว
  - ถ้าคำตอบมากกว่า 1 แสดงว่า class นั้นกำลังแบกรับความรับผิดชอบมากเกินไป
- การแยก responsibility ที่ต่างกันออกเป็นคนละ class ทำให้แต่ละ class มีขอบเขตที่ชัดเจน
  - เมื่อใดที่มีการเปลี่ยนแปลง จะรู้ทันทีว่าต้องไปแก้ที่ class ไหนโดยไม่ต้องเปิดไฟล์อื่นที่ไม่เกี่ยวข้อง
- สัญญาณเตือน 2 แบบที่บ่งบอกว่า class กำลังละเมิด SRP
  - God Class คือ class ที่ทำ “ทุกอย่าง” ในระบบ มักมีชื่อกว้างๆ อย่าง Manager, Handler, System และมี method จำนวนมากที่ไม่เกี่ยวข้องกันเลย
  - Feature Envy คือ method ที่ใช้ข้อมูลของ class อื่นมากกว่าข้อมูลของ class ตัวเอง ซึ่งเป็นสัญญาณว่า method นั้นอาจควรย้ายไปอยู่ใน class ที่มันสนใจจริงๆ มากกว่า

ก่อน Refactor — SRP Violation

พิจารณา class Report ต่อไปนี้ ซึ่งทำงานได้ถูกต้องสมบูรณ์เช่นเดียวกับ UserManager ที่เห็นในส่วนที่ 1

```python
class Report:
    """class นี้มีเหตุผลที่ต้องเปลี่ยนถึงสี่อย่าง"""

    def __init__(self, title: str, data: list):
        self.title = title
        self.data = data

    # Responsibility 1: การคำนวณ (business logic)
    def calculate_summary(self) -> dict:
        return {
            "count": len(self.data),
            "total": sum(self.data),
            "average": sum(self.data) / len(self.data) if self.data else 0
        }

    # Responsibility 2: การ format ผลลัพธ์ (presentation)
    def format_as_text(self) -> str:
        s = self.calculate_summary()
        return f"=== {self.title} ===\nCount: {s['count']}\nTotal:{s['total']}"

    def format_as_csv(self) -> str:
        return "\n".join(str(d) for d in self.data)

    # Responsibility 3: การบันทึกไฟล์ (persistence)
    def save_to_file(self, filename: str) -> None:
        with open(filename, "w") as f:
            f.write(self.format_as_text())

    # Responsibility 4: การแจ้งเตือน (notification)
    def send_by_email(self, recipient: str) -> None:
        print(f"Sending '{self.title}' to {recipient}...")
```

comment ที่กำกับไว้ในแต่ละส่วน คือการ นับเหตุผลในการเปลี่ยนแปลง

ได้ทั้งหมด 4 เหตุผลที่แตกต่างกันโดยสิ้นเชิงใน class เดียว

จำลองสถานการณ์

เพื่อให้เห็นผลกระทบที่วัดได้จริง ลองจำลองว่าแต่ละ requirement เปลี่ยนแยกกัน แล้วดูว่าต้องแก้ไขตรงไหนบ้าง

1. เปลี่ยนสูตรคำนวณ : สมมติบริษัทต้องการเพิ่ม median เข้าไปใน summary ต้องแก้ calculate_summary()
2. เปลี่ยนรูปแบบการแสดงผล : สมมติต้องการ format แบบ HTML เพิ่มเติม ต้องเพิ่ม method ใหม่ในบริเวณเดียวกับ format_as_text() และ format_as_csv()
3. เปลี่ยนที่เก็บข้อมูล : สมมติต้องเปลี่ยนจากบันทึกไฟล์เป็นบันทึกลง cloud storage ต้องแก้ save_to_file()
4. เปลี่ยนช่องทางแจ้งเตือน: สมมติต้องการแจ้งเตือนผ่าน Line แทน email ต้องแก้ send_by_email()

- ทั้งสี่การเปลี่ยนแปลงนี้มาจากทีมงานคนละฝ่ายกันในองค์กรจริง (นักวิเคราะห์ธุรกิจ, ทีม UX, ทีม infrastructure, ทีม marketing)
  - ทุกครั้งต้องเปิดไฟล์เดียวกันคือ report.py
  - ทุกครั้งมีความเสี่ยงที่การแก้ไขของฝ่ายหนึ่งจะ merge conflict หรือกระทบการทำงานของอีกฝ่ายโดยไม่ตั้งใจ แม้เนื้อหาที่แก้จะไม่เกี่ยวข้องกันเลยก็ตาม

หลัง Refactor — แต่ละ Class มี Responsibility เดียว

```python
# Responsibility 1: Business logic — เปลี่ยนเมื่อ calculation rule เปลี่ยน
class Report:
    def __init__(self, title: str, data: list):
        self.title = title
        self.data = data

    def calculate_summary(self) -> dict:
        return {
            "count": len(self.data),
            "total": sum(self.data),
            "average": sum(self.data) / len(self.data) if self.data else 0
        }
```

```python
# Responsibility 2: Presentation — เปลี่ยนเมื่อ format เปลี่ยน
class ReportFormatter:
    def format_as_text(self, report: Report) -> str:
        s = report.calculate_summary()
        return f"==={report.title} ===\nCount: {s['count']}\nTotal:s['total']}"

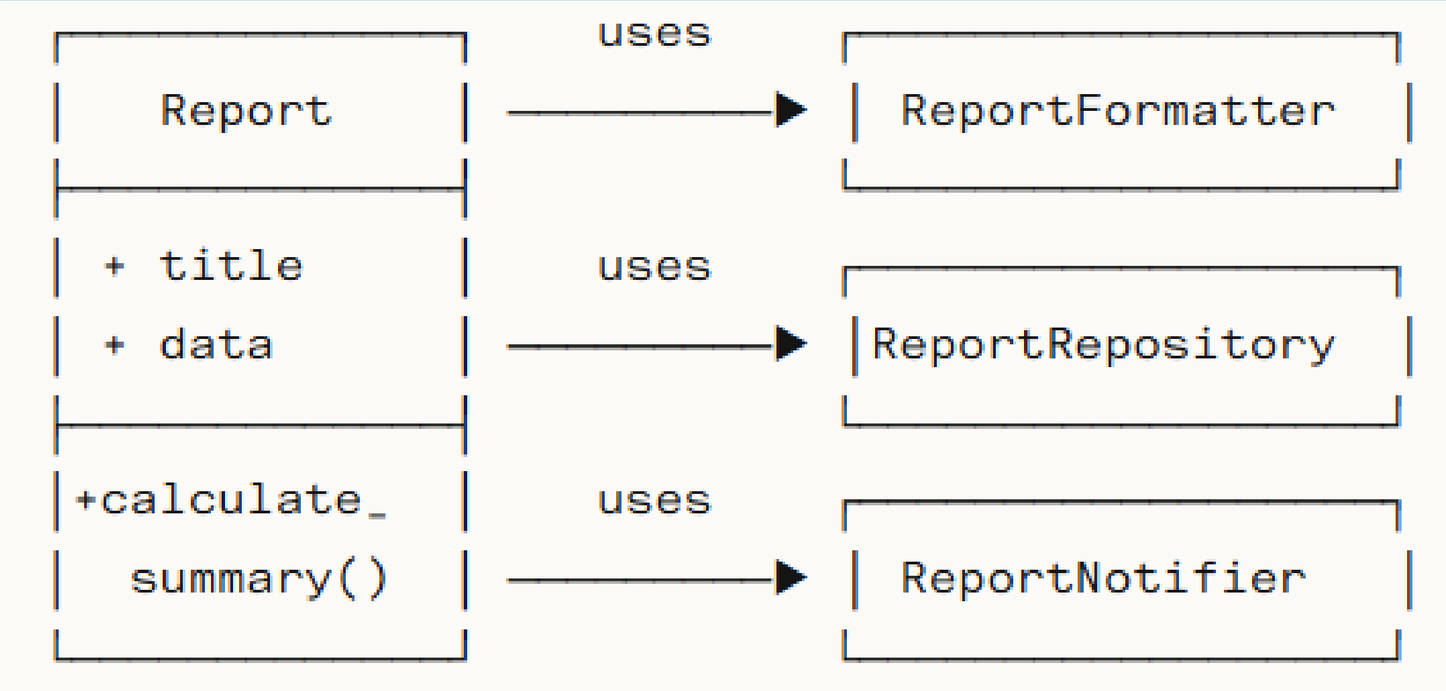
    def format_as_csv(self, report: Report) -> str:
        return "\n".join(str(d) for d in report.data)
```

```python
# Responsibility 3: Persistence — เปลี่ยนเมื่อ storage layer เปลี่ยน
class ReportRepository:
    def save(self, report: Report, filename: str) -> None:
        formatter = ReportFormatter()
        with open(filename, "w") as f:
            f.write(formatter.format_as_text(report))
```

```python
# Responsibility 4: Notification — เปลี่ยนเมื่อ notification channel เปลี่ยน
class ReportNotifier:
    def send_by_email(self, report: Report, recipient: str) -> None:
        print(f"Sending '{report.title}' to {recipient}...")
```

ไม่มีการเปลี่ยนแปลงใดที่กระทบมากกว่าหนึ่งไฟล์อีกต่อไป
ทีมที่ทำงานคนละฝ่ายกันสามารถแก้ไข code คนละไฟล์พร้อมกันได้โดยไม่ชนกันเลย

UML หลัง Refactor



ความสัมพันธ์ระหว่าง class ทั้งสี่เป็นแบบ Association (uses) ไม่ใช่ Composition หรือ Aggregation
เพราะ ReportFormatter, ReportRepository, ReportNotifier ไม่ได้ “เป็นส่วนหนึ่ง” ของ Report ในเชิงโครงสร้าง
แต่เป็นเพียง class ที่ นำ Report ไปใช้งาน ชั่วคราวเพื่อทำหน้าที่เฉพาะของตัวเอง

ประโยชน์ที่ได้จริงในการทำงาน

- ความเสี่ยงต่อ bug ลดลง
  - เพราะแต่ละ class มีขอบเขตแคบ
  - การแก้ไข ReportFormatter มีโอกาสกระทบ ReportRepository น้อยมาก เนื่องจากทั้งสองไม่ได้ปะปนกันในไฟล์เดียวอีกต่อไป
- เวลาในการทำความเข้าใจลดลง
  - เพราะเมื่อต้องแก้ไขเรื่องการบันทึกไฟล์ ผู้พัฒนาเปิดแค่ ReportRepository ที่มีเพียง method เดียวเกี่ยวกับการบันทึก
  - ไม่ต้องไล่อ่านทั้ง class ใหญ่เพื่อแยกส่วนที่เกี่ยวข้องออกมาเหมือนเดิม
- การทดสอบง่ายขึ้นอย่างชัดเจน
  - เพราะสามารถเขียน unit test สำหรับ ReportFormatter โดยไม่ต้อง mock file system หรือ mock email service เลย
  - ทดสอบเฉพาะ logic การ format ได้อย่างโดดเดี่ยวแท้จริง

กับดักที่ต้องระวัง: แยกมากเกินไปก็เป็นปัญหาเช่นกัน

- SRP ไม่ได้แปลว่ายิ่งแยก class มากเท่าไรยิ่งดี
  - ถ้าสิ่งที่จะแยกออกมา ไม่มี reason to change ที่เป็นอิสระจากกันจริง การแยกจะเพิ่ม complexity โดยไม่ได้ประโยชน์อะไรตอบแทน
- ตัวอย่างเช่น การแยก calculate_count(), calculate_total(), calculate_average() ออกเป็นสาม class ต่างหาก ทั้งที่ทั้งสามคำนวณมาจาก self.data เดียวกันและเปลี่ยนแปลงไปด้วยเหตุผลเดียวกันเสมอ (กฎการคำนวณสถิติเปลี่ยน)
  - การแยกแบบนี้ไม่ได้ทำให้ระบบยืดหยุ่นขึ้นเลย กลับทำให้ต้องเปิดไฟล์สามไฟล์เพื่อเข้าใจ logic ที่ควรอยู่ด้วยกัน
- ต้องใช้คำถามเดิมเป็นเกณฑ์เสมอ “ถ้า requirement เปลี่ยน จะต้องแก้ class นี้ด้วยเหตุผลกี่อย่าง”
  - ถ้าสิ่งที่กำลังพิจารณาแยกออกมามีเหตุผลในการเปลี่ยนแปลงเดียวกันกับส่วนที่เหลือ ก็ไม่จำเป็นต้องแยก

OCP (Open/Closed Principle)

OCP (Open/Closed Principle)

- “Software entities should be open for extension, but closed for modification.”
  - “Open for extension” หมายถึง เมื่อระบบต้องการความสามารถใหม่ ต้อง เพิ่มสิ่งใหม่เข้าไปได้
    - ระบบต้องพร้อมรับการเติบโต
  - “Closed for modification” หมายถึง เมื่อเพิ่มความสามารถใหม่นั้น ไม่ควรต้องกลับไปแก้ไข code เดิมที่ทำงานถูกต้องอยู่แล้ว
    - code ที่ผ่านการทดสอบแล้วควรถูกรบกวนให้น้อยที่สุด
- OCP จึงหมายความว่า เมื่อต้องการเพิ่ม functionality ใหม่ ควรทำโดยเพิ่ม code ใหม่ ไม่ใช่แก้ไข code เดิม
- ในทางปฏิบัติ การ “เพิ่มโดยไม่แก้ของเดิม” เป็นไปได้จริง ด้วยกลไก Abstraction ผ่าน Abstract Class
  - สร้าง **“สัญญากลาง”** (abstract method) ขึ้นมาก่อน
  - แล้วให้ทุกความสามารถใหม่ implement **สัญญานั้น** เป็น class แยกต่างหาก
- การเพิ่มความสามารถใหม่จะกลายเป็นการ **สร้าง class ใหม่ที่** implement **สัญญาเดิม**
  - “เพิ่ม” ไฟล์ใหม่เข้าไปในระบบ ไม่ใช่การ “แก้” ไฟล์เดิมที่มีอยู่แล้ว

สัญญาณที่บ่งชี้ OCP Violation

- if-elif chain ที่โตขึ้นเรื่อยๆ คือ สัญญาณ
- ทุกครั้งที่ต้องเพิ่ม elif ใหม่เพื่อรองรับ type ใหม่ นั่นคือสัญญาณว่ากำลังละเมิด OCP
- Shotgun Surgery คือ คำที่ใช้เรียกอาการที่ต้องแก้ไข code หลายจุดกระจัดกระจาย เพื่อรองรับ feature เดียว
  - ยิ่ง feature ใหม่ต้องแก้หลายจุดเท่าไร ยิ่งเสี่ยงลืมแก้บางจุดมากเท่านั้น

```python
class ReportExporter:
    def export(self, data: list, format: str) -> str:
        """เพิ่ม format ใหม่ทุกครั้ง → ต้องแก้ method นี้"""
        if format == "text":
            return "\n".join(str(d) for d in data)
        elif format == "csv":
            return ",".join(str(d) for d in data)
        elif format == "json":
            import json
            return json.dumps(data)
        # ถ้าต้องเพิ่ม XML, PDF, Excel → ต้องแก้ class นี้ทุกครั้ง!
        else:
            raise ValueError(f"Unknown format: {format}")
```

class นี้ทำงานถูกต้องสมบูรณ์สำหรับ 3 รูปแบบที่มีอยู่

แต่ทุกครั้งที่เพิ่ม format ใหม่ ReportExporter ต้องถูก “เปิด” แก้ไข ซึ่งขัดกับหลักการ “closed for modification”

จำลองสถานการณ์: เพิ่ม Format ที่ห้า

ต้องเพิ่ม XMLFormatter เข้าไปในระบบเดิมที่ใช้ if-elif เทียบกับระบบที่ออกแบบตาม OCP

- กรณีที่ 1 — ระบบเดิมที่ใช้ if-elif
  - ต้องเปิดไฟล์ report_exporter.py แล้วเพิ่ม elif format == "xml": เข้าไปในกลาง method ที่มีอยู่แล้ว
  - มีความเสี่ยง 3 ประการ (1) ต้องอ่าน logic เดิมทั้งหมดก่อนเพื่อเข้าใจโครงสร้างแล้วแทรกเงื่อนไขใหม่ให้ถูกตำแหน่ง (2) มีโอกาสพิมพ์ผิดหรือแก้ไขส่วนอื่นโดยไม่ตั้งใจ (3) ต้องรัน regression test ของทุก format เดิมใหม่ทั้งหมด เพราะไม่มีอะไรรับประกันว่าการแก้ไขใน method เดียวกันจะไม่กระทบ format ที่ทำงานดีอยู่แล้ว
- กรณีที่ 2 — ระบบที่ออกแบบตาม OCP
  - การเพิ่ม XML format คือ การสร้างไฟล์ใหม่ XMLFormatter เพียงไฟล์เดียว
  - ไม่แตะไฟล์เดิมแม้แต่บรรทัดเดียว และไม่จำเป็นต้อง regression test format เดิมเลย เพราะ code เดิมไม่ได้ถูกแก้ไข

หลัง Refactor — ปฏิบัติตาม OCP

```python
from abc import ABC, abstractmethod

class Formatter(ABC):
    """Contract ที่ทุก formatter ต้องปฏิบัติตาม"""
    @abstractmethod
    def format(self, data: list) -> str: ...

class TextFormatter(Formatter):
    def format(self, data: list) -> str:
        return "\n".join(str(d) for d in data)

class CSVFormatter(Formatter):
    def format(self, data: list) -> str:
        return ",".join(str(d) for d in data)

class JSONFormatter(Formatter):
    def format(self, data: list) -> str:
        import json
        return json.dumps(data)

class ReportExporter:
    """Closed for modification — ไม่ต้องแก้ class นี้เมื่อเพิ่ม format ใหม่"""
    def export(self, data: list, formatter: Formatter) -> str:
        return formatter.format(data)
```

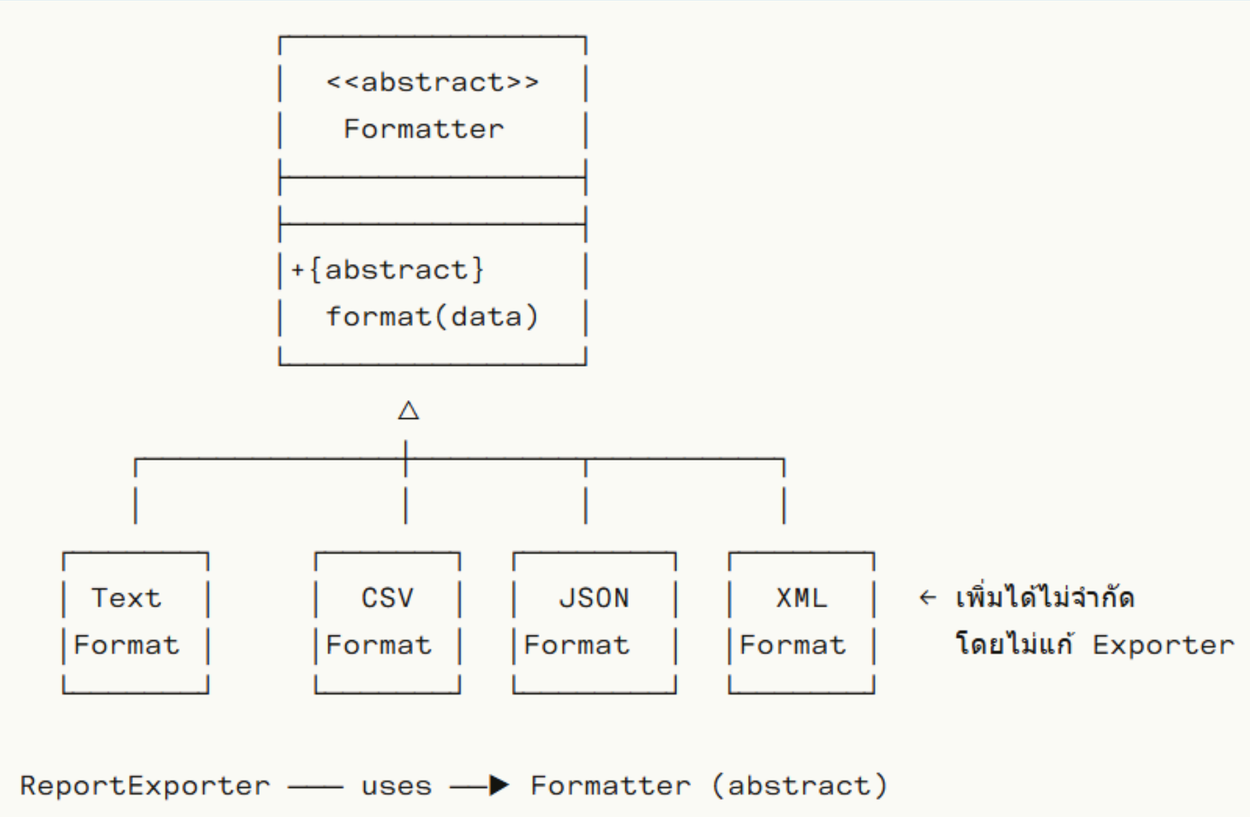
```python
# เพิ่ม XMLFormatter → ไม่ต้องแก้ ReportExporter แม้แต่บรรทัดเดียว
class XMLFormatter(Formatter):
    def format(self, data: list) -> str:
        items = "".join(f"<item>{d}</item>" for d in data)
        return f"<data>{items}</data>"
```

ReportExporter.export() ไม่มีการเช็คว่า formatter เป็น class ไหนเลย มันแค่เรียก formatter.format(data) แล้วปล่อยให้ Polymorphism ทำหน้าที่เลือก implementation ที่ถูกต้องให้เองโดยอัตโนมัติ

การเพิ่ม XMLFormatter เป็นเพียงการเพิ่ม class ใหม่ที่ inherit จาก Formatter เดิม

ReportExporter ที่เขียนไว้ก่อนหน้าไม่ถูกแตะต้องแม้แต่ตัวอักษรเดียว นี่คือความหมายของ “closed for modification, open for extension”

UML ที่แสดง OCP



เส้น ReportExporter ——— uses ———▶ Formatter (abstract)
แสดงให้เห็นว่า ReportExporter รู้จักแค่ “สัญญา” (Formatter) เท่านั้น

ไม่เคยรู้จัก concrete class ตัวใดตัวหนึ่งโดยตรง

ประโยชน์ที่ได้จริงในการทำงาน

- ความเสี่ยงต่อ regression bug ลดลงสำหรับ feature เดิม
  - เมื่อ code เดิมไม่ถูกแตะต้อง ก็ไม่มีทางที่ feature เดิมจะพังจากการเพิ่ม feature ใหม่
  - ต่างจาก if-elif chain ที่ทุกการแก้ไขมีโอกาสกระทบ branch อื่นที่อยู่ในฟังก์ชันเดียวกัน
- พัฒนาแบบขนานได้
  - นักพัฒนาคนหนึ่งสามารถสร้าง XMLFormatter ในขณะที่อีกคนสร้าง PDFFormatter พร้อมกันได้ โดยไม่มีโอกาส merge conflict เนื่องจากทั้งคู่ทำงานในไฟล์คนละไฟล์ที่ไม่แตะกัน
- Third-party หรือทีมอื่นสามารถขยายระบบได้โดยไม่ต้องเข้าถึง source code เดิม
  - แค่รู้จัก interface ของ Formatter ก็สามารถเขียน formatter ของตัวเองมาเสียบเข้าระบบได้ทันที
  - หลักการเดียวกับที่ library และ framework ใหญ่ๆ ใช้ในการออกแบบ plugin system

ข้อควรระวัง

- **แม้ OCP จะมีประโยชน์ชัดเจน แต่ก็ไม่ควรสร้าง Abstract Class ไว้ล่วงหน้าสำหรับทุกสิ่งที่ “อาจจะ” มีหลายแบบในอนาคต**
  - **การสร้าง abstraction ที่ไม่จำเป็นจะเพิ่มความซับซ้อนของระบบโดยไม่ได้ประโยชน์อะไร**
- **หลักปฏิบัติ คือ ถ้าตอนนี้มีแค่หนึ่งรูปแบบและยังไม่เห็นสัญญาณชัดเจนว่าจะมีรูปแบบที่สองในอนาคตอันใกล้ ให้เขียน code แบบตรงไปตรงมาก่อน**
  - refactor เป็น Abstract Class เมื่อเริ่มเห็น if-elif ตัวที่สองหรือสาม เกิดขึ้นจริง

LSP (Liskov Substitution Principle)

SOLID

Common Mistakes



THANK YOU!